

实验 9 单层脉冲神经网络监督学习：Tempotron 与 ReSuMe

1 实验目的

1. 掌握 Tempotron 的二分类决策逻辑：理解输入脉冲如何通过突触后电位核函数累积为膜电位曲线，并由最大膜电位是否超过阈值决定是否发放。
2. 理解 Tempotron 的误分类更新规则：观察正样本未发放与负样本错误发放时，突触权重更新方向为什么相反。
3. 掌握 ReSuMe 的目标脉冲序列学习思想：理解目标脉冲、实际输出脉冲、STDP 与 anti-STDP 在时序监督中的作用。
4. 比较“是否发放”和“何时发放”两类监督目标：通过可视化结果分析 Tempotron 和 ReSuMe 分别适合的任务形式。

2 实验说明

本次实验围绕两个单层脉冲神经网络监督学习算法展开：

1. Tempotron 二分类时空脉冲模式识别：学习“这个输入模式应该让神经元发放，还是不应该发放”。
2. ReSuMe 远程监督学习算法的目标脉冲序列学习：学习“输出神经元应该在什么时刻发放”。

本次实验代码中已经给出数据生成、训练循环和可视化框架，只需要补全最关键的学习机制。本实验共设置 8 处 TODO：

1. 在 Tempotron 实验中补全 PSP 双指数核函数；
2. 在 Tempotron 实验中补全膜电位计算；
3. 在 Tempotron 实验中补全最大膜电位定位与是否发放判断；

4. 在 Tempotron 实验中补全误分类时的权重更新；
5. 在 ReSuMe 实验中补全输入脉冲迹变量；
6. 在 ReSuMe 实验中补全输出神经元仿真；
7. 在 ReSuMe 实验中补全目标脉冲驱动的增强项；
8. 在 ReSuMe 实验中补全实际输出脉冲驱动的抑制项。

在开始实验之前，请先激活实验 1 中创建的虚拟环境，并安装本次实验需要用到的库（已经安装可以跳过）：

```
conda activate brain
pip install numpy matplotlib
```

如果校园网下载速度较慢，也可以使用镜像源，例如：

```
pip install numpy matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple
```

一、Tempotron 的二分类时空脉冲模式识别

Tempotron 是一种电压驱动的单层脉冲神经网络监督学习算法。给定一段输入脉冲序列，目标类别应引发输出神经元发放，非目标类别应保持沉默。

本实验构造一个小型二分类任务：

- 正样本：一组输入神经元构成目标模式，它们在某个较短时间窗内共同发放；这个时间中心在不同样本中会随机变化；
- 负样本：另一组输入神经元构成非目标模式，它们也会共同发放，但这类模式不应该引发输出神经元发放；
- 输出：一个 Tempotron 神经元；
- 分类规则：整段时间内最大膜电位 $V_{\max}$ 超过阈值 V_{th} ，则判为“发放类”；否则判为“不发放类”。

每个样本由整张输入脉冲序列图组成，所有输入脉冲都参与膜电位计算和权重更新。

图中蓝色、橙色、灰色分别表示目标模式输入、非目标模式输入和其他输入。
Tempotron 使用整段时间内的最大膜电位 $V_{\max}$ 进行分类。

1.1 PSP 核函数与膜电位

Tempotron 膜电位可写为：

$$V(t) = V_{rest} + \sum_i w_i \sum_{t_i^f} K(t - t_i^f) \quad (1)$$

其中， t_i^f 表示第 i 个输入神经元的某一次脉冲发放时间， w_i 是对应突触权重， $K(t - t_i^f)$ 表示该输入脉冲引起的突触后电位响应。

本实验使用双指数 PSP 核函数：

$$K(t - t_i^f) = V_0 \left(e^{-(t-t_i^f)/\tau_m} - e^{-(t-t_i^f)/\tau_s} \right) H(t - t_i^f) \quad (2)$$

其中， $H(t - t_i^f)$ 表示只有当 $t \geq t_i^f$ 时，该输入脉冲才会影响膜电位。代码中将核函数峰值归一化到约 1，相当于选取对应的 V_0 。

分类时需要找出：

$$t_{\max} = \operatorname{argmax}_t V(t) \quad (3)$$

$$V_{\max} = V(t_{\max}) \quad (4)$$

若 $V_{\max} \geq V_{th}$ ，则认为神经元发放；否则认为神经元不发放。

1.2 Tempotron 误分类更新

Tempotron 只在误分类时更新权重：

- 正样本应该发放，但实际没有发放：增强在 $t_{\max}$ 前对膜电位有贡献的输入；
- 负样本不应该发放，但实际错误发放：削弱在 $t_{\max}$ 前对膜电位有贡献的输入；
- 分类正确：不更新。

本实验采用下面的离散更新形式：

$$\Delta w_i = \begin{cases} +\eta \sum_{t_i^f < t_{\max}} K(t_{\max} - t_i^f), & \text{正样本未发放} \\ -\eta \sum_{t_i^f < t_{\max}} K(t_{\max} - t_i^f), & \text{负样本错误发放} \\ 0, & \text{分类正确} \end{cases} \quad (5)$$

更新只作用于 $t_{\max}$ 前对膜电位峰值有贡献的输入。

1.3 实验代码

请将下面代码保存为 `exp9_1.py`，补全其中的 TODO 1 到 TODO 4 后运行。

```
import os
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = [
    "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei", "SimHei",
    "Noto Sans CJK SC", "Arial Unicode MS", "DejaVu Sans",
]
plt.rcParams["axes.unicode_minus"] = False

# 本实验要求修改的参数
THRESHOLD = 1.0          # 发放阈值：越高越容易漏掉正样本，越低越容易让负样本
                           # 误发放
ETA = 0.025              # Tempotron 学习率；控制每次误分类后的权重更新幅度
PATTERN_JITTER = 2.0     # 同一模式内共同发放的时间抖动；越大表示共同发放越不
                           # 精确

# 其余默认参数（一般不需要修改）
N_INPUTS = 24
T_MAX = 80
TAU_M = 15.0
TAU_S = 3.0
KEY_NEURONS = np.arange(0, 8)
NEGATIVE_PATTERN_NEURONS = np.arange(8, 16)
DISTRACTOR_NEURONS = np.arange(16, N_INPUTS)
NON_KEY_NEURONS = np.arange(8, N_INPUTS)
POS_CENTER_RANGE = (15, 65)
```

```

def psp_kernel(delta_t, tau_m=TAU_M, tau_s=TAU_S):
    # TODO 1:
    # 请补全 Tempotron 使用的双指数 PSP 核函数。
    # 要求:
    # 1. 当  $\text{delta\_t} < 0$  时, 核函数值应为 0;
    # 2. 当  $\text{delta\_t} \geq 0$  时, 计算  $\exp(-\text{delta\_t}/\text{tau\_m}) - \exp(-\text{delta\_t}/\text{tau\_s})$ ;
    # 3. 将核函数除以其最大值, 使峰值约为 1。
    raise NotImplementedError("请补全 psp_kernel")

def generate_temporal_patterns(num_per_class=60, seed=7):
    rng = np.random.default_rng(seed)
    samples = []
    labels = []

    for _ in range(num_per_class):
        spikes = np.zeros((N_INPUTS, T_MAX), dtype=float)
        center_t = int(rng.integers(POS_CENTER_RANGE[0], POS_CENTER_RANGE[1] + 1))

        for neuron in KEY_NEURONS:
            t = int(np.clip(rng.normal(loc=center_t, scale=PATTERN_JITTER), 5, T_MAX - 5))
            spikes[neuron, t] = 1.0
            for _ in range(10):
                neuron = int(rng.choice(NON_KEY_NEURONS))
                t = int(rng.integers(5, T_MAX - 5))
                spikes[neuron, t] = 1.0
            samples.append(spikes)
            labels.append(1)

    for _ in range(num_per_class):
        spikes = np.zeros((N_INPUTS, T_MAX), dtype=float)
        center_t = int(rng.integers(POS_CENTER_RANGE[0], POS_CENTER_RANGE[1] + 1))
        for neuron in NEGATIVE_PATTERN_NEURONS:
            t = int(np.clip(rng.normal(loc=center_t, scale=PATTERN_JITTER), 5, T_MAX - 5))

```

```

        spikes[neuron, t] = 1.0
    for _ in range(10):
        neuron = int(rng.choice(np.concatenate([KEY_NEURONS, DISTRA
        CTOR_NEURONS])))
        t = int(rng.integers(5, T_MAX - 5))
        spikes[neuron, t] = 1.0
    samples.append(spikes)
    labels.append(0)

samples = np.array(samples)
labels = np.array(labels, dtype=int)
order = rng.permutation(len(labels))
return samples[order], labels[order]

```

```

def compute_voltage(spikes, weights, v_rest=0.0):
    # TODO 2:

    # 请根据  $V(t)=V_{rest}+\sum_i w_i \sum_{\{t_i^f\}} K(t-t_i^f)$  计算整段膜电位
    曲线。

    # 提示:

    # 1. voltage 的形状应为 (T_MAX,);
    # 2. 需要遍历每个输入神经元;
    # 3. 对同一个输入神经元的多个脉冲也要逐个累加 PSP;
    # 4. 不同输入神经元的贡献需要按权重加权后相加。

    raise NotImplementedError("请补全 compute_voltage")

```

```

def classify_by_peak(voltage, threshold):
    # TODO 3:

    # 请找出最大膜电位时刻 t_max、最大膜电位 v_max，并判断是否越过阈值。
    # 返回 predicted_spike, t_max, v_max。

    raise NotImplementedError("请补全 classify_by_peak")

```

```

def tempotron_weight_update(spikes, label, predicted_spike, t_max, eta):
    # TODO 4:

    # 请补全 Tempotron 误分类更新规则。

```

```

# 说明:
# 1. label=1 表示正样本, 应该发放;
# 2. label=0 表示负样本, 不应该发放;
# 3. 正样本未发放时, direction=+1;
# 4. 负样本错误发放时, direction=-1;
# 5. 分类正确时, 返回全 0 的 delta_w;
# 6. 只累加  $t_i^f < t_{max}$  的输入脉冲贡献。
raise NotImplementedError("请补全 tempotron_weight_update")

```

```

def evaluate(samples, labels, weights, threshold):
    preds = []
    for spikes in samples:
        voltage = compute_voltage(spikes, weights)
        pred, _, _ = classify_by_peak(voltage, threshold)
        preds.append(pred)

    preds = np.array(preds, dtype=int)
    positive_mask = labels == 1
    negative_mask = labels == 0

    error_rate = float(np.mean(preds != labels))
    positive_hit_rate = float(np.mean(preds[positive_mask] == 1))
    false_alarm_rate = float(np.mean(preds[negative_mask] == 1))

    return {
        "error_rate": error_rate,
        "positive_hit_rate": positive_hit_rate,
        "false_alarm_rate": false_alarm_rate,
    }

```

```

def train_tempotron(samples, labels, epochs=35, eta=ETA, threshold=THRESHOLD, seed=11):
    rng = np.random.default_rng(seed)
    # 初始权重设置为未学会状态: 目标模式输入偏低, 非目标模式输入偏高。
    weights = rng.uniform(0.03, 0.08, size=N_INPUTS)
    weights[KEY_NEURONS] = rng.uniform(0.01, 0.04, size=len(KEY_NEURONS))
    weights[NEGATIVE_PATTERN_NEURONS] = rng.uniform(0.12, 0.20, size=len(

```

```

n(NEGATIVE_PATTERN_NEURONS))
    initial_weights = weights.copy()
    history = {"error_rate": [], "positive_hit_rate": [], "false_alarm_
rate": []}

    for _ in range(epochs):
        order = rng.permutation(len(labels))
        for sample_idx in order:
            spikes = samples[sample_idx]
            label = int(labels[sample_idx])
            voltage = compute_voltage(spikes, weights)
            pred, t_max, _ = classify_by_peak(voltage, threshold)
            delta_w = tempotron_weight_update(spikes, label, pred, t_max, eta)
            weights = np.clip(weights + delta_w, 0.0, 1.2)

        metrics = evaluate(samples, labels, weights, threshold)
        history["error_rate"].append(metrics["error_rate"])
        history["positive_hit_rate"].append(metrics["positive_hit_rate"])
        history["false_alarm_rate"].append(metrics["false_alarm_rate"])

    return weights, initial_weights, history

```

```

def plot_input_spike_sequence(ax, spikes, title):
    event_times = [np.where(spikes[i] > 0)[0] for i in range(spikes.shape[0])]
    colors = []
    for i in range(spikes.shape[0]):
        if i in KEY_NEURONS:
            colors.append("#2563eb")
        elif i in NEGATIVE_PATTERN_NEURONS:
            colors.append("#f97316")
        else:
            colors.append("#666666")
    ax.eventplot(event_times, lineoffsets=np.arange(spikes.shape[0]), linewidths=0.75, colors=colors)
    ax.set_xlim(0, T_MAX)
    ax.set_title(title)
    ax.set_xlabel("时间 (ms)")
    ax.set_ylabel("输入神经元")
    ax.grid(axis="x", alpha=0.18)
    ax.text(
        0.01,

```



```

        0.98,
        "蓝：目标模式输入    橙：非目标模式输入    灰：其他输入",
        transform=ax.transAxes,
        va="top",
        fontsize=9,
        color="#333333",
        bbox=dict(boxstyle="round,pad=0.25", facecolor="#ffffff", edgec
olor="#dddddd", alpha=0.85),
    )

```

```

def plot_voltage(ax, spikes, weights_before, weights_after, threshold,
title):
    before = compute_voltage(spikes, weights_before)
    after = compute_voltage(spikes, weights_after)
    pred_b, t_b, v_b = classify_by_peak(before, threshold)
    pred_a, t_a, v_a = classify_by_peak(after, threshold)
    ax.plot(before, label=f"训练前 Vmax={v_b:.2f}, pred={pred_b}", colo
r="#94a3b8", linewidth=2.0)
    ax.plot(after, label=f"训练后 Vmax={v_a:.2f}, pred={pred_a}", color
="#c1121f", linewidth=2.2)
    ax.axhline(threshold, color="#333333", linestyle="--", linewidth=1.
2, label="阈值")
    ax.scatter([t_b], [v_b], color="#64748b", s=42, zorder=3)
    ax.scatter([t_a], [v_a], color="#c1121f", s=52, zorder=3)
    ax.set_xlim(0, T_MAX)
    ax.set_title(title)
    ax.set_xlabel("时间 (ms)")
    ax.set_ylabel("膜电位")
    ax.grid(alpha=0.2)
    ax.legend(fontsize=9, loc="upper right")

```

```

def main():
    samples, labels = generate_temporal_patterns(num_per_class=70, seed
=7)
    threshold = THRESHOLD
    weights, initial_weights, history = train_tempotron(samples, labels,
threshold=threshold)
    metrics = evaluate(samples, labels, weights, threshold)

    print("final error rate:", metrics["error_rate"])
    print("positive hit rate:", metrics["positive_hit_rate"])

```

```
print("false alarm rate:", metrics["false_alarm_rate"])
```

```
pos_idx = int(np.where(labels == 1)[0][0])  
neg_idx = int(np.where(labels == 0)[0][0])
```

```
fig = plt.figure(figsize=(14, 10))  
gs = fig.add_gridspec(3, 2, height_ratios=[1.0, 1.05, 0.95])  
ax1 = fig.add_subplot(gs[0, 0])  
ax2 = fig.add_subplot(gs[0, 1])  
ax3 = fig.add_subplot(gs[1, 0])  
ax4 = fig.add_subplot(gs[1, 1])  
ax5 = fig.add_subplot(gs[2, :])
```

plot_input_spike_sequence(ax1, samples[pos_idx], "正样本输入脉冲序列
图：目标模式输入共同发放")

plot_input_spike_sequence(ax2, samples[neg_idx], "负样本输入脉冲序列
图：非目标模式输入共同发放")

plot_voltage(ax3, samples[pos_idx], initial_weights, weights, thres
hold, "正样本膜电位曲线")

plot_voltage(ax4, samples[neg_idx], initial_weights, weights, thres
hold, "负样本膜电位曲线")

ax5.bar(np.arange(N_INPUTS) - 0.18, initial_weights, width=0.36, la
bel="训练前", color="#94a3b8")

ax5.bar(np.arange(N_INPUTS) + 0.18, weights, width=0.36, label="训
练后", color="#15803d")

ax5.axvspan(KEY_NEURONS[0] - 0.5, KEY_NEURONS[-1] + 0.5, color="#db
eafe", alpha=0.7, label="目标模式输入")

```
ax5.axvspan(  
    NEGATIVE_PATTERN_NEURONS[0] - 0.5,  
    NEGATIVE_PATTERN_NEURONS[-1] + 0.5,  
    color="#ffedd5",  
    alpha=0.7,  
    label="非目标模式输入",  
)
```

```
ax5.set_title("训练前后的突触权重")
```

```
ax5.set_xlabel("输入神经元")
```

```
ax5.set_ylabel("突触权重")
```

```
ax5.grid(axis="y", alpha=0.2)
```

```

ax5.legend(fontsize=9)

fig.suptitle("Tempotron 二分类时空脉冲模式识别", fontsize=18, weight=
"bold")
fig.tight_layout(rect=[0, 0, 1, 0.96])
os.makedirs("outputs", exist_ok=True)
fig.savefig("outputs/exp9_1_tempotron_result.png", dpi=180)
plt.show()

if __name__ == "__main__":
    main()

```

1.4 参数修改与结果分析

请先补全 `psp_kernel`、`compute_voltage`、`classify_by_peak` 和 `tempotron_weight_update`，再运行整个脚本。

补全后，重点修改以下参数：

- **THRESHOLD**：观察阈值升高或降低时，正样本漏发和负样本误发如何变化；
- **ETA**：观察学习率过小、适中、过大时，突触权重和分类结果的变化；
- **PATTERN_JITTER**：观察目标模式输入或非目标模式输入的共同发放越不精确时，膜电位峰值和分类结果如何变化。

在报告中分析以下问题：

1. 观察正样本和负样本的输入脉冲序列图。二者是否都存在一组输入神经元在短时间窗内共同发放？这说明类别差异主要来自“脉冲数量”还是“特定通道的时序结构”？
2. 观察训练前后的突触权重图。目标模式输入、非目标模式输入和其他输入的权重分别发生了什么变化？这些权重变化如何解释正负样本膜电位曲线中的 $V_{\max}$ 变化？
3. 分别调大或调小 **THRESHOLD**、**ETA**、**PATTERN_JITTER**。哪些设置会对正负样

本的判别产生影响？哪些设置会对学习产生影响？分别产生何种影响？请结合相关图像观察现象并解释。

二、ReSuMe 的目标脉冲序列学习

Tempotron 解决“是否发放”的监督分类问题。ReSuMe (Remote Supervised Method, 远程监督学习算法) 学习指定的输出发放时刻。本实验采用单输出神经元和输入脉冲迹变量实现。

本实验设置一个单输出神经元的时序监督任务。

- 输入：一组固定输入脉冲序列，其中包含目标脉冲前输入、非目标时刻输入和其他输入；
- 目标输出：在 $20ms, 32ms, 44ms, 56ms, 68ms, 80ms$ 发放；
- 实际输出：由当前突触权重和 LIF/SRM 式膜电位动态生成；
- 初始权重：目标脉冲前输入权重偏低，容易漏掉目标脉冲；非目标时刻输入权重偏高，容易在非目标时刻错位发放；
- 学习目标：让实际输出脉冲从训练前的错位发放，逐渐靠近目标输出脉冲序列。

ReSuMe 的更新由目标脉冲和实际输出脉冲的差异决定：

- 目标脉冲出现而实际输出没有出现时，增强近期活跃输入对应的突触；
- 实际输出出现而目标脉冲没有出现时，抑制近期活跃输入对应的突触。

2.1 ReSuMe 权重更新

ReSuMe 权重更新形式为：

$$\frac{d}{dt}w_{oi}(t) = [S_d(t) - S_o(t)] \left[a_d + \int_0^\infty a_{di}(s)S_i(t-s)ds \right] \quad (6)$$

其中， $S_d(t)$ 表示目标脉冲序列， $S_o(t)$ 表示实际输出脉冲序列， $S_i(t)$ 表示第 i 个

输入神经元的输入脉冲序列。积分项表示目标脉冲或实际输出脉冲之前的输入脉冲历史。

代码中用离散输入脉冲迹变量近似这个历史项：

$$x_i^{trace}(t) = \exp(-1/\tau_{trace}) x_i^{trace}(t-1) + x_i(t) \quad (7)$$

其中， $x_i(t)$ 表示第 i 个输入神经元在时刻 t 是否发放， τ_{trace} 控制迹变量的衰减速度。

离散实现中，根据目标脉冲和实际输出脉冲是否一致来更新权重：

- 目标脉冲出现、实际输出没有出现：增强近期活跃输入；
- 实际输出出现、目标脉冲没有出现：抑制近期活跃输入；
- 二者同时出现或同时不出现：不更新。

$$\Delta w_i = \begin{cases} +\eta_d x_i^{trace}(t), & S_d(t) = 1, S_o(t) = 0 \\ -\eta_o x_i^{trace}(t), & S_d(t) = 0, S_o(t) = 1 \\ 0, & S_d(t) = S_o(t) \end{cases} \quad (8)$$

2.2 输出神经元模型

本实验使用简化的离散 LIF/SRM 形式：

$$u(t) = \alpha u(t-1) + \sum_i w_i x_i(t) \quad (9)$$

当 $u(t) \geq V_{th}$ 时，输出神经元发放一个脉冲，并把膜电位复位。

2.3 实验代码

请将下面代码保存为 exp9_2.py，补全其中的 TODO 5 到 TODO 8 后运行。

```
import os
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = [
    "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei", "SimHei",
    "Noto Sans CJK SC", "Arial Unicode MS", "DejaVu Sans",
```

```

]
plt.rcParams["axes.unicode_minus"] = False

# 本实验要求修改的参数
ETA_TARGET = 0.012      # 目标脉冲驱动的增强幅度；越大越容易补上缺失目标脉冲
ETA_OUTPUT = 0.012      # 实际输出脉冲驱动的抑制幅度；越大越容易压制多余或错位输出
TAU_TRACE = 6.0         # 输入脉冲迹变量时间常数；越大表示更久以前的输入也会被归因

# 其余默认参数（一般不需要修改）
THRESHOLD = 0.95
ALPHA = 0.84
N_INPUTS = 32
T_MAX = 100
TARGET_TIMES = np.array([20, 32, 44, 56, 68, 80])
TARGET_INPUT_GROUPS = [np.arange(i * 3, (i + 1) * 3) for i in range(len(TARGET_TIMES))]
TARGET_INPUTS = np.concatenate(TARGET_INPUT_GROUPS)
NON_TARGET_TIMES = np.array([26, 38, 50, 62, 74])
NON_TARGET_INPUTS = np.arange(18, 30)
OTHER_INPUTS = np.arange(30, N_INPUTS)

def generate_input_spikes(seed=13):
    rng = np.random.default_rng(seed)
    spikes = np.zeros((N_INPUTS, T_MAX), dtype=float)

    for group, target_t in zip(TARGET_INPUT_GROUPS, TARGET_TIMES):
        for neuron in group:
            for loc in [target_t - 2, target_t]:
                t = int(np.clip(rng.normal(loc=loc, scale=0.45), 5, T_MAX - 5))
                spikes[neuron, t] = 1.0

    for idx, neuron in enumerate(NON_TARGET_INPUTS):
        non_target_t = NON_TARGET_TIMES[idx % len(NON_TARGET_TIMES)]
        for loc in [non_target_t - 2, non_target_t]:
            t = int(np.clip(rng.normal(loc=loc, scale=0.5), 5, T_MAX - 5))
            spikes[neuron, t] = 1.0

```

```

for neuron in OTHER_INPUTS:
    for _ in range(2):
        t = int(rng.integers(8, T_MAX - 8))
        spikes[neuron, t] = 1.0

```

```

return spikes

```

```

def generate_target_spikes():
    target = np.zeros(T_MAX, dtype=float)
    target[TARGET_TIMES] = 1.0
    return target

```

```

def compute_pre_trace(input_spikes, tau_trace=TAU_TRACE):
    # TODO 5:
    # 请补全输入脉冲迹变量。
    # 要求:
    # 1. trace 的形状与 input_spikes 相同;
    # 2. 每个时间步先让上一时刻迹变量按  $\exp(-1/\tau\_trace)$  衰减;
    # 3. 再加上当前时间步的输入脉冲;
    # 4. 返回每个输入神经元在每个时刻的迹变量。
    trace = np.zeros_like(input_spikes, dtype=float)
    raise NotImplementedError("请补全 compute_pre_trace")

```

```

def simulate_output_neuron(input_spikes, weights, alpha=ALPHA, threshold=THRESHOLD, reset=0.0):
    # TODO 6:
    # 请补全输出神经元仿真。
    # 要求:
    # 1. 使用  $u(t)=\alpha*u(t-1)+\sum_i w_i*x_i(t)$ ;
    # 2. 当  $u(t)\geq threshold$  时,  $output[t]=1$ ;
    # 3. 发放后将膜电位复位为 reset;
    # 4. 返回 membrane 和 output, 二者形状均为  $(T\_MAX,)$ 。

```

```

raise NotImplementedError("请补全 simulate_output_neuron")

def resume_update(input_spikes, target_spikes, output_spikes, eta_target=ETA_TARGET, eta_output=ETA_OUTPUT, tau_trace=TAU_TRACE):
    pre_trace = compute_pre_trace(input_spikes, tau_trace=tau_trace)
    delta_w = np.zeros(input_spikes.shape[0], dtype=float)

    for t in range(T_MAX):
        if target_spikes[t] > 0 and output_spikes[t] == 0:
            # TODO 7:
            # 目标脉冲出现而实际输出没有出现时，根据输入脉冲迹变量增强对应突触。

            # 近期活跃输入应更容易推动目标时刻发放。
            raise NotImplementedError("请补全 ReSuMe 的目标脉冲增强项")

        if output_spikes[t] > 0 and target_spikes[t] == 0:
            # TODO 8:
            # 实际输出脉冲出现而目标脉冲没有出现时，根据输入脉冲迹变量执行 anti-STDP 抑制。

            # 实际输出出现时，削弱近期活跃输入。
            raise NotImplementedError("请补全 ReSuMe 的实际输出抑制项")

    return delta_w

def nearest_spike_distance(target_spikes, output_spikes, tolerance=2):
    target_times = np.where(target_spikes > 0)[0]
    output_times = np.where(output_spikes > 0)[0]

    if len(output_times) == 0:
        return {"missed": len(target_times), "extra": 0, "mean_distance": float(T_MAX)}

    distances = []
    missed = 0
    for target_t in target_times:

```



```

    nearest_idx = int(np.argmin(np.abs(output_times - target_t)))
    nearest_t = int(output_times[nearest_idx])
    distance = abs(nearest_t - target_t)
    distances.append(distance)
    if distance > tolerance:
        missed += 1

extra = 0
for out_t in output_times:
    if np.min(np.abs(target_times - out_t)) > tolerance:
        extra += 1

return {"missed": int(missed), "extra": int(extra), "mean_distance":
float(np.mean(distances))}

def train_resume(input_spikes, target_spikes, epochs=120, seed=5):
    rng = np.random.default_rng(seed)
    # 初始权重设置为未学会状态: 目标脉冲前输入偏低, 非目标时刻输入偏高。
    weights = rng.uniform(0.01, 0.03, size=N_INPUTS)
    for group in TARGET_INPUT_GROUPS:
        weights[group] = rng.uniform(0.015, 0.035, size=len(group))
    weights[NON_TARGET_INPUTS] = rng.uniform(0.22, 0.28, size=len(NON_T
TARGET_INPUTS))
    initial_weights = weights.copy()
    snapshots = {}

    for epoch in range(epochs + 1):
        membrane, output = simulate_output_neuron(input_spikes, weights)
        if epoch in {0, epochs}:
            snapshots[epoch] = {"weights": weights.copy(), "membrane":
membrane.copy(), "output": output.copy()}

        if epoch == epochs:
            break

    delta_w = resume_update(input_spikes, target_spikes, output)
    weights = np.clip(weights + delta_w, 0.0, 1.2)

    return weights, initial_weights, snapshots

```

```

def main():
    input_spikes = generate_input_spikes(seed=13)
    target_spikes = generate_target_spikes()
    final_weights, initial_weights, snapshots = train_resume(input_spikes, target_spikes)
    final_membrane, final_output = simulate_output_neuron(input_spikes, final_weights)
    metrics = nearest_spike_distance(target_spikes, final_output)

    print("target spike times:", np.where(target_spikes > 0)[0].tolist())
    print("initial output spike times:", np.where(snapshots[0]["output"] > 0)[0].tolist())
    print("final output spike times:", np.where(final_output > 0)[0].tolist())
    print("missed target spikes:", metrics["missed"])
    print("extra output spikes:", metrics["extra"])
    print("mean target-to-output distance:", metrics["mean_distance"])

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    events = [np.where(input_spikes[i] > 0)[0] for i in range(N_INPUTS)]
    colors = []
    for i in range(N_INPUTS):
        if i in TARGET_INPUTS:
            colors.append("#2563eb")
        elif i in NON_TARGET_INPUTS:
            colors.append("#f97316")
        else:
            colors.append("#666666")
    axes[0, 0].eventplot(events, lineoffsets=np.arange(N_INPUTS), linewidths=0.72, colors=colors)
    axes[0, 0].set_title("输入脉冲序列图：目标脉冲前输入与非目标时刻输入")
    axes[0, 0].set_xlim(0, T_MAX)
    axes[0, 0].set_xlabel("时间 (ms)")
    axes[0, 0].set_ylabel("输入神经元")
    axes[0, 0].grid(axis="x", alpha=0.18)
    axes[0, 0].text(0.01, 0.96, "蓝：目标脉冲前输入    橙：非目标时刻输入\n灰：其他输入", transform=axes[0, 0].transAxes, va="top", fontsize=9)

    before = snapshots[0]
    after = snapshots[max(snapshots.keys())]
    rows = [np.where(target_spikes > 0)[0], np.where(before["output"] > 0)[0], np.where(after["output"] > 0)[0]]

```

```

    axes[0, 1].eventplot(rows, lineoffsets=[2, 1, 0], linelengths=0.65,
        colors=["#c1121f", "#94a3b8", "#2563eb"])
    axes[0, 1].set_yticks([2, 1, 0])
    axes[0, 1].set_yticklabels(["目标输出", "训练前实际输出", "训练后实际
输出"])
    axes[0, 1].set_xlim(0, T_MAX)
    axes[0, 1].set_title("目标脉冲与实际输出脉冲对比")
    axes[0, 1].set_xlabel("时间 (ms)")
    axes[0, 1].grid(axis="x", alpha=0.2)

    axes[1, 0].plot(before["membrane"], color="#94a3b8", linewidth=2.0,
        label="训练前膜电位")
    axes[1, 0].plot(after["membrane"], color="#2563eb", linewidth=2.2,
        label="训练后膜电位")
    axes[1, 0].axhline(THRESHOLD, color="#333333", linestyle="--", line
width=1.2, label="阈值")
    for target_t in np.where(target_spikes > 0)[0]:
        axes[1, 0].axvline(target_t, color="#c1121f", linestyle=":", li
newwidth=1.2)
    axes[1, 0].set_xlim(0, T_MAX)
    axes[1, 0].set_title("训练前后膜电位曲线")
    axes[1, 0].set_xlabel("时间 (ms)")
    axes[1, 0].set_ylabel("膜电位")
    axes[1, 0].grid(alpha=0.2)
    axes[1, 0].legend(fontsize=9)

    x = np.arange(N_INPUTS)
    axes[1, 1].bar(x - 0.18, initial_weights, width=0.36, color="#94a3b
8", label="训练前")
    axes[1, 1].bar(x + 0.18, final_weights, width=0.36, color="#15803d",
        label="训练后")
    axes[1, 1].axvspan(TARGET_INPUTS[0] - 0.5, TARGET_INPUTS[-1] + 0.5,
        color="#d8eafe", alpha=0.65, label="目标脉冲前输入")
    axes[1, 1].axvspan(NON_TARGET_INPUTS[0] - 0.5, NON_TARGET_INPUTS[-1]
+ 0.5, color="#ffedd5", alpha=0.7, label="非目标时刻输入")
    axes[1, 1].set_title("训练前后的突触权重")
    axes[1, 1].set_xlabel("输入神经元")
    axes[1, 1].set_ylabel("突触权重")
    axes[1, 1].grid(axis="y", alpha=0.2)

```

```
axes[1, 1].legend(fontsize=8)

fig.suptitle("ReSuMe 目标脉冲序列学习", fontsize=18, weight="bold")
fig.tight_layout(rect=[0, 0, 1, 0.96])
os.makedirs("outputs", exist_ok=True)
fig.savefig("outputs/exp9_2_resume_result.png", dpi=180)
plt.show()
```

```
if __name__ == "__main__":
    main()
```

2.4 参数修改与结果分析

请先补全 `compute_pre_trace`、`simulate_output_neuron` 和 `resume_update`，再运行整个脚本。

补全后，重点修改以下参数：

- **ETA_TARGET**：观察目标脉冲出现而实际输出缺失时，增强幅度变强或变弱会如何影响漏发；
- **ETA_OUTPUT**：观察实际输出出现而目标脉冲缺失时，抑制幅度变强或变弱会如何影响多余或错位输出；
- **TAU_TRACE**：观察输入脉冲迹变量保留时间变长或变短后，哪些输入更容易被归因到目标脉冲或实际输出脉冲。

在报告中分析以下问题：

1. 观察目标输出、训练前实际输出和训练后实际输出的脉冲序列图。训练前实际输出主要表现为？训练后这些现象分别有什么变化？
2. 观察训练前后的突触权重图。目标脉冲前输入、非目标时刻输入和其他输入的权重分别发生了什么变化？这些变化如何解释实际输出脉冲向目标脉冲靠近的现象？
3. 分别调大或调小 **ETA_TARGET**、**ETA_OUTPUT**、**TAU_TRACE**。它们会对输出脉冲

的发放和学习产生何种影响？请结合目标/实际输出图、膜电位图和权重图观察并解释相关现象。

3 实验要求

1. 完成实验说明中的 2 个主线实验：
 - Tempotron 二分类时空脉冲模式识别；
 - ReSuMe 目标脉冲序列学习。
2. 补全文中留出的 8 处 TODO，保证代码能够正常运行。
3. 至少提交 2 组实验结果图，需覆盖以下内容：
 - Tempotron 实验中的正负样本输入脉冲序列图、训练前后膜电位曲线和训练前后的突触权重图；
 - ReSuMe 实验中的输入脉冲序列图、目标/实际输出对比图、训练前后膜电位曲线和训练前后的突触权重图。
4. 提交实验代码文件。
5. 提交一份实验报告或结果说明，报告中必须包含关键图像、参数设置、现象分析，并回答各实验小节和下方列出的问题。

实验报告中回答以下问题：

1. Tempotron 的膜电位由所有输入脉冲经过 PSP 核函数加权累积得到。为什么 Tempotron 使用整段膜电位曲线中的最大值 $V_{\max}$ 作为分类依据？若两个样本的总脉冲数接近、也都有短时间共同发放，Tempotron 仍可能区分它们。请结合其学习方式解释为什么。
2. ReSuMe 的公式含有 $S_d(t) - S_o(t)$ 。当目标脉冲和实际输出脉冲同时出现时，为什么不应该继续增强或抑制？当只出现目标脉冲或只出现实际输出脉冲时，权重更新分别在修正什么错误？

3. 思考 ReSuMe 对“错位但很接近”的输出脉冲处理是否合理。若网络在 $30ms$ 发放，而目标脉冲在 $32ms$ ，代码会在这两个时刻分别怎样更新权重？由此你认为 ReSuMe 是否存在一些局限性？

提交内容如下：

- 实验代码文件，命名为 `exp9_1.py`、`exp9_2.py`；
- 一份实验报告；
- 报告中应包含关键图像、参数设置以及对实验现象的分析。